

Deep Learning

Object Detection with KerasCV

G. Marques

KerasCV at a Glance

KERASCV extends Keras with computer-vision building blocks not available in the base library.

- **Detection models** — `YOLOV8Detector`, `RetinaNet` with pre-trained backbones.
- **Detection-aware augmentation** — layers that transform bounding boxes together with images (`JitteredResize`, `RandomFlip`, ...).
- **Bounding-box utilities** — format conversion, IoU computation, `pad_batch`.
- **COCO evaluation metrics** — `BoxCOCOMetrics` reports mAP, mAP@50, mAP@75 after each epoch.
- **Pre-trained weights** — downloaded via `from_preset()` (backbone trained on COCO 80 classes).

Keras team at Google. KerasCV: A computer vision extension library for Keras.

<https://github.com/keras-team/keras-cv>, 2023.

YOLOV8Detector — Architecture

Three stacked components:

- **BACKBONE (CSPDARKNET)** — extracts feature maps at multiple scales (strides 8, 16, 32).
- **NECK (FPN)** — fuses multi-scale features; depth controlled by `fpn_depth`.
- **DETECTION HEAD** — *anchor-free*; predicts a bounding box and class scores per spatial location.

Key innovation over YOLO v5: *decoupled head* (separate branches for box regression and classification) and no predefined anchor boxes.

Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLO.
<https://github.com/ultralytics/ultralytics>, 2023.

Version 8.0

YOLOV8Detector — Architecture

Available backbone presets (COCO pre-trained):

- `'yolo_v8_xs_backbone_coco'` — extra-small, fastest
- `'yolo_v8_s_backbone_coco'` — small
- `'yolo_v8_m_backbone_coco'` — medium (*used in this lab*)
- `'yolo_v8_l_backbone_coco'` — large, most accurate

Transfer-learning strategy: the backbone keeps COCO weights; the detection head is *re-initialised* with random weights and trained from scratch on GTSDB.

Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLO.
<https://github.com/ultralytics/ultralytics>, 2023.

Version 8.0

Bounding Box Formats in KerasCV

All KerasCV layers share a `bounding_box_format` parameter. Choose one format and keep it consistent across the pipeline.

Format	Values	Notes
<code>xyxy</code>	$[x_1, y_1, x_2, y_2]$	Absolute pixels, top-left + bottom-right
<code>xywh</code>	$[x, y, w, h]$	Top-left corner + size
<code>rel_xyxy</code>	$[x_1, y_1, x_2, y_2]$	Normalised $[0, 1]$
<code>center_xywh</code>	$[c_x, c_y, w, h]$	Centre + size

Annotations are passed as a Python **dict**:

- `'boxes'` — tensor of shape $(B, N, 4)$.
- `'classes'` — tensor of shape (B, N) , integer labels.

Pick `'xyxy'` for GTSDb — annotations in `gt.txt` are already in $[x_1, y_1, x_2, y_2]$ absolute-pixel form.

Code: Bounding Box Format Conversion

Use `keras_cv.bounding_box.convert_format()` to change format at any pipeline stage.

```
1 import keras_cv, tensorflow as tf
2
3 BBOX_FMT = 'xyxy'
4
5 # Bounding boxes as a dict (batch of 1 image, 1 box)
6 boxes = {
7     'boxes': tf.constant([[[10., 20., 90., 80.]]]),
8     'classes': tf.constant([[0]])
9 }
10
11 # Convert xywh -> xyxy
12 boxes_xyxy = keras_cv.bounding_box.convert_format(
13     boxes,
14     source='xywh',
15     target='xyxy',
16     images=images      # required for relative formats
17 )
18 print(boxes_xyxy['boxes']) # [[[10., 20., 100., 100.]]]
```

GTSDDB — 4 Super-classes

The German Traffic Sign Detection Benchmark [3] has **43 fine-grained classes** grouped into 4 super-classes:

Label	Super-class	Examples
0	Prohibitory	Speed limits, no overtaking, no trucks
1	Danger	Bends, crossings, slippery road, animals
2	Mandatory	Keep right/left, roundabout, go straight
3	Other	Priority road, give way, stop, no entry

- Annotation file: `gt.txt` — one line per bounding box.
- Format: `filename;x1;y1;x2;y2;class_id` (43 fine classes).
- Class IDs do *not* follow contiguous ranges per super-class — always derive the mapping from `ReadMe.txt`.

Sebastian Houben, Johannes Stallkamp, Jan Salmen, Marc Schlipf, and Christian Igel. Detection of traffic signs in real-world images: The German Traffic Sign Detection Benchmark.

In *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2013

Data Pipeline — Key Steps

Building a KerasCV-compatible detection dataset from raw PNG files:

- 1 **Parse annotations** — read `gt.txt`, group boxes by image, map 43 classes → 4 super-classes.
- 2 **Load images** — `tf.io.read_file` + `tf.image.decode_png`, then resize to (384, 640).
- 3 **Build bounding-box dict** — `{'boxes': ..., 'classes': ...}` in 'xyxy' format.
- 4 **Batch** — fixed batch size; pad boxes to the same length per image (use -1 as the padding class).
- 5 **Augment** (train only) — detection-aware layers transform boxes together with images.
- 6 **Prefetch** — `.prefetch(tf.data.AUTOTUNE)` overlaps data loading with GPU computation.

Code: Parse `gt.txt` into Python Lists

`boxes_arr[i]: float32 array shape (Ni, 4), Ni ∈ [0,6]. Variable length ⇒ list of arrays ⇒ ragged tensors for tf.data.`

```
1 import pandas as pd, numpy as np
2 import glob
3 # 0=Prohibitory 1=Danger 2=Mandatory 3=Other
4 # float32 (not int): KerasCV requires classes as float32
5 _MAP = np.array([0,0,0,0,0,0,3,0,0,0,0,1,3,3,3,0,0,3,1,1,1,1,1,
6                 1,1,1,1,1,1,1,1,3,2,2,2,2,2,2,2,2,3,3], 'f4')
7
8 gt = pd.read_csv('gt.txt', sep=';',
9                 names=['filename', 'x1', 'y1', 'x2', 'y2', 'class_id'])
10 gt['filename'] = gt['filename'].str.replace('.ppm', '.png')
11
12 img_paths, boxes_arr, classes_arr = [], [], []
13 for path in sorted(glob.glob('GTSDB/*.png')):
14     name = path.split('/')[1]
15     rows = gt[gt.filename == name]
16     img_paths.append(path)
17     if len(rows) == 0: # sign-free image
18         boxes_arr.append(np.zeros((0,4), 'f4'))
19         classes_arr.append(np.zeros((0,), 'f4'))
20     else:
21         boxes_arr.append(
22             rows[['x1', 'y1', 'x2', 'y2']].values.astype('f4'))
23         classes_arr.append(_MAP[rows['class_id'].values])
```

Image Sizes and the Stride Constraint

Detector	Default size	Constraint
YOLOv3 / v4	416×416	mult. of 32
YOLOv5 / v7 / v8	384×640	mult. of 32
SSD300 / SSD512	300×300 / 512×512	fixed
RetinaNet	~800×1333 (variable)	mult. of 128
EfficientDet D0–D4	512–1024	mult. of 128
Faster R-CNN	~800×1333 (variable)	mult. of 32

- Each strided conv / pooling stage halves the feature map: k stages $\Rightarrow$ stride = 2^k .
YOLOv8 / SSD: 5 stages $\Rightarrow$ stride 32 — both input dims must be $\equiv 0 \pmod{32}$, else fractional feature maps cause shape errors in the neck.
- RetinaNet FPN adds P6, P7 (strides 64, 128) $\Rightarrow$ inputs ideally divisible by 128.
- Images need *not* be square: 384×640 (both $\div 32$) suits GTSDb's 1.7:1 ratio — only 8 px padding vs. 264 px for 640×640.

Code: Building the `tf.data` Pipeline

Pattern: `map` → `cache` → `shuffle` → `ragged_batch`: decode once, cache in RAM (≈ 2.6 GB), reshuffle each epoch. **Remove `.cache()` if memory-limited.**

`pad_to_aspect_ratio=True`: keeps aspect ratio; pads short side gray — preserves sign shape.

```
1 import tensorflow as tf, keras_cv
2
3 BBOX_FMT = 'xyxy'
4 IMG_SIZE = (640, 384)
5 BATCH = 8
6 letterbox = keras_cv.layers.Resizing(*IMG_SIZE,
7     pad_to_aspect_ratio=True, bounding_box_format=BBOX_FMT)
8
9 def load_sample(img_path, boxes, classes):
10     img = tf.cast(tf.image.decode_png(
11         tf.io.read_file(img_path), channels=3), 'float32') / 255.0
12     return img, {'boxes': boxes, 'classes': classes}
13 def apply_lb(imgs, bb):
14     d = letterbox({'images': imgs, 'bounding_boxes': bb})
15     return d['images'], d['bounding_boxes']
16
17 ds = (tf.data.Dataset.from_tensor_slices((img_paths,
18     tf.ragged.constant(boxes_arr), # (N_i, 4) per image
19     tf.ragged.constant(classes_arr) # (N_i,) per image
20 ))
21     .map(load_sample, num_parallel_calls=tf.data.AUTOTUNE)
22     .cache() # ~2.6 GB RAM; remove if memory-limited
23     .shuffle(1000) # 1000 > 900: full shuffle each epoch
24     .ragged_batch(BATCH) # variable #boxes/image -> RaggedTensor batch
25     .map(apply_lb, num_parallel_calls=tf.data.AUTOTUNE)
26     .prefetch(tf.data.AUTOTUNE))
```

Code: Detection-Aware Augmentation

Geometric layers (flip, resize) take the full KerasCV dict and adjust boxes. Colour layers act on the image tensor only — boxes are unaffected.

```
1 import keras_cv
2
3 flip_layer = keras_cv.layers.RandomFlip(
4     mode='horizontal', bounding_box_format=BBOX_FMT)
5 resize_layer = keras_cv.layers.JitteredResize(
6     target_size=IMG_SIZE, scale_factor=(0.75, 1.3),
7     bounding_box_format=BBOX_FMT)
8 brightness = keras_cv.layers.RandomBrightness(
9     factor=0.2, value_range=(0, 1))
10 contrast = keras_cv.layers.RandomContrast(
11     factor=0.2, value_range=(0, 1))
12
13 def augment(images, bboxes):
14     data = {'images': images, 'bounding_boxes': bboxes}
15     data = flip_layer(data)
16     data = resize_layer(data)
17     data['images'] = brightness(data['images'])
18     data['images'] = contrast(data['images'])
19     return data['images'], data['bounding_boxes']
20
21 ds_train = ds_train.map(
22     augment, num_parallel_calls=tf.data.AUTOTUNE)
```

Note: omit `flip_layer` for the 43-class task — a horizontal flip turns a “turn right” sign into a “turn left” (a different class).

Loading a Pre-trained Backbone

`from_preset()` downloads CSPDarknet backbone weights trained on COCO 80-class detection.

- The backbone starts with *rich, general-purpose* visual features.
- The detection head (FPN + prediction layers) is **randomly initialised** — it must be trained from scratch on GTSDb.
- `num_classes=4` sets up a 4-class output head; GTSDb uses 4 super-classes.
- `fpn_depth=2` adds two FPN fusion stages between the backbone and the detection head.

Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLO.
<https://github.com/ultralytics/ultralytics>, 2023.

Version 8.0

Code: Building the YOLOV8Detector

Combine the pre-trained backbone with a fresh 4-class detection head.

```
1 import keras_cv
2
3 backbone = keras_cv.models.YOLOV8Backbone.from_preset(
4     'yolo_v8_m_backbone_coco'
5 )
6
7 model = keras_cv.models.YOLOV8Detector(
8     num_classes=4,
9     bounding_box_format=BBOX_FMT,
10    backbone=backbone,
11    fpn_depth=2
12 )
13
14 model.summary()
15 # Backbone : ~25 M params (COCO pre-trained)
16 # FPN + Head: ~3 M params (randomly initialised)
```

Phase 1 — Why Freeze the Backbone?

In Phase 1 we train **only the detection head**; the backbone weights are frozen (`trainable = False`).

- The head is randomly initialised — its gradients are *large and noisy* at the start of training.
- Letting those noisy gradients flow into the backbone would **corrupt the pre-trained feature representations**.
- With the backbone frozen, far fewer parameters are updated $\Rightarrow$ faster epochs and lower overfitting risk on a small dataset like GTSDb.
- Once the head has converged, Phase 2 unfreezes the backbone for joint fine-tuning (at a much lower learning rate).

Code: Freeze the Backbone

Set `trainable = False` on the backbone before compiling.

```
1 import tensorflow as tf
2
3 # Freeze backbone weights
4 model.backbone.trainable = False
5
6 # Report the parameter split
7 n_train = sum(tf.size(w).numpy()
8               for w in model.trainable_weights)
9 n_total = sum(tf.size(w).numpy()
10              for w in model.weights)
11
12 print(f'Trainable_{n_train}>12,}')
13 print(f'Total_{n_total}>12,}')
14 # Trainable :      3,145,728
15 # Total      :     28,573,184
```

Only the detection head trains in Phase 1 ($\approx 11\%$ of parameters).

Loss Functions

BOX REGRESSION LOSS: CIoU

Complete IoU penalises three quantities simultaneously:

- **Overlap** — standard IoU between predicted and GT box.
- **Centre distance** — normalised Euclidean distance between box centres.
- **Aspect-ratio consistency** — penalises predicted boxes whose width/height ratio differs from the GT.

Better than L1/L2 on raw coordinates: *scale-invariant* and directly optimises the IoU evaluation metric.

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\text{dist}_{\text{euc}}^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v$$
$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2, \quad \alpha = \frac{v}{(1 - \text{IoU}) + v}$$

c : diagonal of the smallest enclosing box covering both predicted and GT boxes.

Loss Functions

CLASSIFICATION LOSS: BINARY CROSS-ENTROPY

Each class is treated as an *independent binary* prediction:

- Output: one sigmoid probability per class per spatial location.
- Target: one-hot vector, e.g. $[0, 0, 1, 0]$ for “danger”.
- *No softmax* — allows multi-label detection if needed.

Both losses are specified as strings in `model.compile()`:

```
box_loss='ciou'  
classification_loss='binary_crossentropy'
```

Code: Phase 1 — Compile (Head Only)

Freeze the backbone and compile before the first training phase.

```
1 import tensorflow as tf
2
3 model.backbone.trainable = False
4
5 model.compile(
6     optimizer=tf.keras.optimizers.Adam(
7         learning_rate=1e-3),
8     box_loss='ciou',
9     classification_loss='binary_crossentropy',
10    jit_compile=False      # disable XLA for compatibility
11 )
```

`learning_rate=1e-3` is safe here because the head weights are randomly initialised — they need a relatively large gradient step.

Code: Phase 1 — Train the Head

Run `model.fit()` to train the detection head for 15–20 epochs.

```
1 ckpt_cb = tf.keras.callbacks.ModelCheckpoint(  
2     'best_yolov8_gtsdb.keras',  
3     monitor='val_loss',  
4     save_best_only=True,  
5     verbose=1)  
6  
7 history1 = model.fit(  
8     ds_train,  
9     epochs=15,  
10    validation_data=ds_val,  
11    callbacks=[ckpt_cb]  
12 )
```

Stop when `val_loss` stops improving. The head should converge before we unlock the backbone.

Phase 2 — Full Fine-Tune

After Phase 1 the head is well-initialised — its gradients are now *small and stable*. We can safely fine-tune **all layers**.

- Unfreeze the backbone: `model.backbone.trainable = True`.
- **Always recompile** after changing `trainable` status — Keras must rebuild the gradient tape.
- Use a **much smaller learning rate** ($\approx 100\times$ smaller than Phase 1) to avoid overwriting pre-trained features.
- Run for 20–30 more epochs; monitor `val_loss` for early stopping.

Code: Phase 2 — Unfreeze and Fine-Tune

Unfreeze the backbone, recompile with a lower LR, and continue training.

```
1 # Unfreeze all backbone layers
2 model.backbone.trainable = True
3
4 # Recompile -- required after changing trainable status
5 model.compile(
6     optimizer=tf.keras.optimizers.Adam(
7         learning_rate=1e-5),      # 100x smaller than Phase 1
8     box_loss='ciou',
9     classification_loss='binary_crossentropy',
10    jit_compile=False
11 )
12
13 history2 = model.fit(
14     ds_train,
15     epochs=20,
16     validation_data=ds_val,
17     callbacks=[ckpt_cb]
18 )
```

COCO Evaluation Metrics

`keras_cv.metrics.BoxCOCOMetrics` computes the standard COCO detection suite after each epoch:

Metric	Description
MaP	mean AP over $\text{IoU} \in [0.50, 0.55, \dots, 0.95]$
MaP@.50IoU	AP at $\text{IoU} \geq 0.50$ (same as PASCAL VOC)
MaP@.75IoU	AP at $\text{IoU} \geq 0.75$ (strict localisation)
MaP-S	AP for small objects (area $< 32^2$ px)
MaP-M	AP for medium objects
MaP-L	AP for large objects

- MaP is the primary COCO leaderboard metric.
- Monitor MaP@.50IoU first — more lenient, reaches useful values earlier in training.
- Large objects (signs close to the camera) typically have higher MaP-L; check MaP-S for distant signs.

Code: Inference

`model.predict()` applies NMS internally and returns a dict with boxes, confidence scores, and class labels.

```
1 # Load best checkpoint before inference
2 model = tf.keras.models.load_model(
3     'best_yolov8_gtsdb.keras')
4
5 # Fetch one validation batch
6 images, gt_boxes = next(iter(ds_val))
7
8 # detections: dict with 'boxes', 'confidence', 'classes'
9 detections = model.predict(images, verbose=0)
10
11 print(detections['boxes'].shape)      # (B, N, 4)
12 print(detections['confidence'].shape) # (B, N)
13 print(detections['classes'].shape)    # (B, N)
14 # N = max detections per image (default 100)
15 # Unused slots: class = -1, confidence ~ 0
```


Code: Visualisation

`keras_cv.visualization.plot_bounding_box_gallery` draws GT and predicted boxes side by side for visual inspection.

```
1 CLASS_MAP = {0: 'prohibitory', 1: 'mandatory',
2              2: 'danger',      3: 'other'}
3
4 keras_cv.visualization.plot_bounding_box_gallery(
5     images[:4],
6     y_true=gt_boxes,
7     y_pred=detections,
8     bounding_box_format=BBOX_FMT,
9     class_mapping=CLASS_MAP,
10    scale=4,
11    rows=2,
12    cols=2
13 )
```

Each panel: ground-truth boxes (left), predicted boxes (right), labelled with class name and confidence score.